

Response to Reviewers

Manuscript JSA-D-26-01708 (transferred to the Journal of Parallel and Distributed Computing)
Streaming LLM Weights from CXL Memory: Regimes, Placement and Pipeline-Aware Pinning
(previous title: *CAMP: Content-Aware Memory Prefetching for High-Performance CXL-Based Inference*)

Quang-Vinh Dang, British University Vietnam — October 2026

We thank both reviewers for detailed and constructive reports. Their main concerns were (i) an implausible hardware path, (ii) an emulator whose curves looked synthetic, (iii) an algorithmic contribution that reduced to frequency-based pinning, and (iv) an evaluation on stylised workloads and a tiny model. We agree with the substance of these points. Rather than patching the old evaluation we rebuilt it: the manuscript now has a new system model, a new simulator whose link and overlap logic is checked against a real GPU, a reformulated pinning algorithm, and a new evaluation on Llama-2/3 and Gemma-2 configurations with prefill, decode and mixed iterations. Section, figure and table numbers below refer to the revised manuscript. Where the revision does not fully resolve a point, or where the new, stronger evaluation weakened a claim of the original submission, we say so.

Summary of what changed.

- Related work reduced from about five pages to about one and a half (Section 2); PIM, distributed training and fault-tolerant DSM material removed; known-graph swapping systems for training and recent offloading work added.
- New host-mediated GPU–CXL path model (Section 3.1, Fig. 1, Table 2); the GPU does not address the Type-3 device.
- New event-driven simulator with roofline compute, link parameters anchored in published measurements and noise; its link and overlap logic is checked on a real GPU (Section 5.2, Table 4, Fig. 2), with the limits of that check stated.
- Algorithm 2 replaced by stall-cost-aware pinning (SCP) with an exact problem statement (Eqs. 6–7); baselines strengthened (equal pinning budget, tuned lookahead, a FlexGen-style per-layer split with tuned buffers) and an attribution study and an exhaustive-optimum comparison added (Sections 6.2–6.4).
- **Headline claims were revised downwards.** The strongest baseline, a tuned FlexGen-style per-layer split, is within a few per cent of the new policy on dense uniform models; the policy’s distinctive gains are on non-uniform workloads (Section 6.5), and the compute-time estimate turns out to matter little (Section 6.6). The abstract, introduction and conclusion now say so.
- Workloads: prefill/decode/mixed iterations on 7B–405B configurations, KV-feasible batching, and comparisons with quantisation, host DRAM and two GPUs (Sections 6.1–6.8); distilgpt2 and the stylised scenarios removed.

Reviewer 1

R1.1 — Section 2 is bloated and partly irrelevant.

Section 2 spans over 5 pages and reads like a generic survey; PIM, distributed training and fault-tolerant DSM have no relevance to weight prefetching.

Response. Agreed. Section 2 is now about a page and a half with four paragraphs: how a GPU reaches CXL memory, weight offloading for LLM inference, known-graph memory management, and positioning (Table 1). PIM/near-data computing, distributed training and serving, fault-tolerant

DSM and generic characterisation material were removed. The remaining text is limited to work that bears on weight streaming; training-time swapping systems (vDNN, SwapAdvisor, AutoTM, Capuchin, Sentinel) are included because they establish known-graph prefetching, and we state that we do not claim that idea as new.

R1.2 — Hardware architecture (Fig. 1): GPUs cannot be directly attached to CXL Type-3 devices.

Type-3 devices are CXL.mem endpoints mapped into the host CPU’s memory space; current GPUs cannot control them without host-mediated PCIe transactions or CXL switches.

Response. The reviewer is right and the old figure was misleading. Fig. 1 is redrawn and Section 3.1 states the path explicitly: the expander is a CPU-less NUMA node; a host runtime thread page-locks the CXL-backed region and issues `cudaMemcpyAsync` on a copy stream; the GPU copy engine reads through the root complex; kernels wait on CUDA events. The effective bandwidth is bounded by the slower of the CXL device and the PCIe link (Eq. 1). We cite measurements showing this behaviour on real devices (a plain copy goes through a bounce buffer; page-locking the region enables direct DMA; 10–18 GB/s achieved) and model the bounce-buffer (“staged”) variant, whose cost we also quantify (Section 6.7: $2.7\text{--}6.7\times$ slower). H100 appears only as the compute model of the GPU. GPU-initiated direct access (NVLink-C2C, TMA-based) and CXL switches are different paths and are listed as not modelled (Sections 2, 3.1, 7).

R1.3 — Limited novelty; similar to PowerInfer.

Algorithm 1 only issues async prefetches while cumulative transfer fits the compute duration; Algorithm 2 is an offline greedy fractional knapsack on static counts, similar to PowerInfer’s hot/cold partition.

Response. We agree that the original algorithms were thin, and the revised evaluation shows that the reviewer’s concern was larger than we first acknowledged. (a) The prefetcher is now a work-conserving controller with next-use admission (Algorithm 1, Section 4.2); the old compute-gated rule never binds in our workloads and gives identical results (Section 6.6). (b) Pinning is now SCP (Section 4.3): its objective is the predicted iteration time of a pipeline planner, solved by plan-and-verify. (c) We implemented prefix-residency, per-layer-split (FlexGen-style), frequency hot/cold (PowerInfer-style), and stride baselines in the same simulator, with equal pinning budgets and tuned lookahead. *Result:* a FlexGen-style per-layer split with tuned buffers ties the new policy within 3.2% on dense uniform models; the conventional hot/cold baseline is up to $1.62\times$ slower; and an attribution study (Section 6.3, Fig. 5) shows that a large part of the gain over a conventional hot/cold baseline comes from *interleaving* the resident layers rather than from the planner (the planner step is comparably large in one of the nine rows). (d) What we now claim instead is a regime analysis and lower bound (Sections 3.3–3.4, 6.1), the finding that placement of resident layers decides performance when compute and transfer are comparable (contiguous sets idle the link; Section 6.4), a policy that matches the best simple rule without tuning and wins where reuse or unit sizes are non-uniform (Section 6.5: up to $1.97\times$ over interleaving-only baselines on encoder–decoder inference, $1.07\times$ over the tuned split on Gemma-2), and a simulator whose link and overlap logic is checked on hardware. We acknowledge that the contribution is an engineering-level policy and analysis rather than a new algorithmic principle, and the title and abstract no longer present the policy as the sole contribution.

R1.4 — Algorithm 2’s reuse formulation degenerates without tied weights.

$F(v)$ is uniform across regular layers; the algorithm degenerates to arbitrary static ordering and should consider true reuse distance or DAG depth.

Response. Agreed, and the new text says so explicitly. (a) SCP does not rank layers by $F(v)$; it minimises the predicted iteration time over pin sets, which depends on position in the sequence, layer sizes and the overlap of compute with transfer (Eqs. 6–9). (b) Transient layers are evicted by true next-use distance (Section 4.3, Algorithm 1). (c) We evaluate three workloads where $F(v)$ genuinely differs (tied embedding, speculative decoding, encoder–decoder; Table 9, Fig. 7). Frequency awareness is decisive for the encoder–decoder (interleaving-only baselines are $1.97\times$ slower), matters little for speculative decoding ($1.06\times$ for the FlexGen-style split) and, for the tied embedding, SCP is $1.07\times$ faster than the tuned FlexGen-style split. In the encoder–decoder case the simplest reuse-aware rule (equal-budget frequency pinning) is as good as SCP; SCP’s contribution is that one mechanism handles all three cases. (d) For uniform dense models the frequency rule degenerates exactly as the reviewer says; there the benefit of any good rule comes from placement (Section 6.4), not frequency.

R1.5 — Stage definition in Section 4.2 is unrealistic.

Thinking/streaming/thrashing scenarios do not map onto prefill and decode.

Response. The three scenarios were removed. Workloads are now prefill passes, decode steps and chunked-prefill mixed iterations (Sections 3.2, 5.3), built from real model configurations with roofline compute (Eq. 2, Table 3). The regime map (Fig. 3) shows how tokens per iteration, which is what distinguishes these phases, determine whether streaming is viable.

R1.6 — Emulation accuracy: curves look synthetic (linear in hit ratio, in depth; abrupt peak).

Fig. 4 is linear in hit ratio, Fig. 8(a) perfectly linear in depth, Fig. 8(c) has an abrupt peak; these weaken the accuracy of the emulation.

Response. The old curves were products of uniform synthetic layers and a single-server queue, and we have replaced that evaluation. In the new simulator, compute follows a roofline model for each layer type and batch descriptor, the link has a per-copy fixed cost, chunking, noise and slow bandwidth drift, and cache effects arise from real layer sizes and the KV-cache-limited cache capacity. Results are no longer smooth: latency versus model size has a knee where the model stops fitting (Fig. 10), the KV cache of a large decode batch shrinks the weight cache and bends the throughput curve (Fig. 9), and the cache-share sweep is non-monotonic (Fig. 11c; worst point $1.07\times$ the lower bound). The comparisons are stable under an $8\times$ increase of the noise level (Section 6.9, Fig. 12a), and a mis-specified link model changes latency by at most 1.3% (Fig. 12c).

R1.7.1 — distilgpt2 (312 MB) is insufficient. Response. Agreed; distilgpt2 was removed. The evaluation uses Llama-2-7B/13B, Llama-3-8B/70B/405B and Gemma-2-9B configurations with public hyper-parameters (Table 3); Llama-3-70B and 405B are evaluated with the HBM that remains after the KV cache of the workload, and Fig. 10 spans 7B to 405B. We do not run these models’ weights; the simulator tracks bytes and times, which is stated in Section 5.1.

R1.7.2 — Graph-aware pinning ties with LFU; does CAMP offer zero algorithmic gain?

Response. The tie was an artefact of the only layer with a frequency difference (the tied embedding). Table 8 now isolates eviction on a uniform decode loop: LRU, FIFO and LFU are identical with zero hits (LFU has no signal on a cyclic scan), random eviction gets 7%, next-use eviction 33%. Table 9 and

Section 6.5 show when frequency information helps and when SCP adds to it. We state plainly that on dense uniform models the gain of SCP over the best simple interleaved rule is small (Sections 6.2, 6.4, 7).

R1.7.3 — Batch scaling: 257.3 to 263.0 tokens/s indicates a miscalibrated compute model. Response. Agreed; that experiment was removed. In the new model weight traffic is independent of batch size and compute follows the roofline. Fig. 9a varies the tokens per iteration and shows linear growth up to about 16k tokens where throughput saturates at the compute-bound limit. Fig. 9b is the realistic decode case, where a larger batch also needs a larger KV cache: the cached share of the weights falls from 54% at $B = 1$ to 23% at $B = 256$, and $B = 512$ no longer fits.

R1.7.4 — No cycle-accurate simulator or hardware counters. Response. We did not use gem5: no cycle-accurate model of an H100 with a CXL Type-3 device exists at the scale needed, and CXL-DMSim-style simulators do not model GPU kernels (Section 7). Instead we checked the simulator’s link and overlap logic against hardware: a CUDA streaming loop on a Tesla T4 (pinned host memory standing in for the CXL pool, so no CXL device was measured) with measured compute-only and copy-only components, composed in the simulator without fitting and compared with 30 measured composite configurations (Section 5.2, Table 4, Fig. 2). The result is mixed and we report it as such: 0.6% mean error in link-bound configurations (maximum 1.3%), 7.6% over all configurations, up to 25% where compute and copy are comparable, with a sign that differs between regimes and an unexplained cause; and a closed-form expression with the same inputs has a mean error of 6.4%, so the check validates the inputs and overlap semantics rather than the simulator’s extra machinery. The measured pageable/pinned bandwidth ratio (0.37) is consistent with the bounce-buffer penalty reported for CXL regions. We also check the engine against closed-form pipeline times in a test suite and the pin planner against the engine (0.18% mean error over 80 random configurations). The remaining limitation is that no CXL hardware, PCIe Gen5 GPU or H100 was measured; this is stated in Sections 1, 5.2 and 7.

Reviewer 2

R2.1 — Eq. (2) is a 0–1 knapsack but Algorithm 2 sorts by reuse frequency. Response. The reviewer is correct that the two were not equivalent. Section 4.3 now separates three objects: (i) the SCP problem (Eq. 6), minimising predicted iteration time subject to a capacity constraint that includes the streaming slot; this is not a knapsack because the objective is not additive and depends on placement; (ii) the knapsack proxy (Eq. 7), whose benefit per unit size is $F(u)t_{\text{copy}}(s_u)/s_u \approx F(u)/BW_{\text{eff}}$, solved exactly by dynamic programming and used as a baseline; (iii) the old frequency-ordered greedy, which is the special case for uniform sizes. We compare all three, plus random, prefix, stride and marginal-greedy sets (Fig. 6, Section 6.4) and, on an 18-unit model, the exhaustive optimum: SCP is 0.71% above it on average (maximum 4.2%) against up to 32.4% for frequency ranking and 20.3% for stride (Table 7). SCP remains a heuristic without a guarantee, and in the event engine its set is up to 12.6% slower than the planner-optimal set.

R2.2 — $T_{\text{comp}}(v_i)$ is required but the paper claims no profiling. Response. Section 4.1 now answers this. $\hat{t}_u$ is computed from tensor shapes and the live batch descriptor with a roofline using generic device-class constants and the peak rates reported by the driver, and is calibrated online by an exponential moving average of observed/nominal duration per layer kind (Eq. 5). The layer’s own duration is not needed before it completes. This removes offline profiling of the model or workload but still needs a probe iteration to calibrate. Sensitivity is quantified in Section 6.6 (Tables 10–11, Fig. 8):

the generic estimate has 23.2% error for prefill and one probe iteration reduces it to at most 1.6%; however, the latency of the resulting plan differs by at most 0.4% between the generic, calibrated and exact estimators, so the estimate matters only when it is systematically biased, and we no longer present the cost model as a central contribution. A shape-dependent ground truth that the estimator cannot represent leaves up to 13% error after calibrating on a different shape and changes latency by at most 0.4%.

R2.3 — Evidence for Graph-Aware Pinning is weak; it ties with LFU. Response. See R1.3, R1.4 and R1.7.2. In addition, the revised evaluation shows where the pin rule matters: contiguous (prefix or frequency-ranked) pin sets leave the link idle (63–80% utilisation in 13B prefill, versus 100–98% for interleaved sets) when compute and transfer are comparable, and an attribution study isolates which ingredients matter (Sections 6.3–6.4, Figs. 5–6, Eq. 10). We also report where there is no difference (link-bound decode, compute-bound large prefill) and where a simple baseline is as good (a tuned FlexGen-style split on dense uniform models; frequency pinning on the encoder–decoder).

R2.4 — The CXL execution path in the emulator needs a concrete description. Response. See R1.2. Section 3.1 states who initiates the transfer (host runtime thread), how data reaches HBM (DMA from page-locked CXL-backed memory through the root complex), the synchronisation (CUDA events), and the modelled overheads (per-copy API/driver/first-byte cost, per-chunk cost, event synchronisation cost, bandwidth bound by the slower link, noise and drift, staged bounce-buffer variant); Table 2 lists the values and their basis. Not modelled: CXL switches, interleaving of several expanders (folded into the effective bandwidth, swept from 8 to 64 GB/s), GPU-initiated direct access, coherence traffic.

Other changes

- The previous multi-tenant and synchronised-burst serving experiments were removed because they depended on the old single-queue model and could not support the claims made from them.
- The abstract, introduction and conclusion were rewritten to state limitations and negative results.
- New comparisons with weight quantisation, pinned host DRAM and two GPUs (Section 6.8), a cost-of-re-planning estimate (Section 7), and an itemised list of limitations (Section 7).
- Code, raw results and the hardware micro-benchmark are in the public repository referenced in the Data availability statement; every table, figure and number in the text is generated from the result files.
- The generative-AI declaration required by the journal is included in the manuscript.